

Marwadi University

BCA Semester - 3

**05BC3301 – Database Management System – 2
(DBMS-2)**

Unit – 5: Triggers

A solid orange horizontal bar spanning the width of the slide, located at the bottom.

Unit – 5: Triggers

5	Database Triggers: Introduction, Types of triggers, Creating DML Triggers (Row trigger, Statement triggers, Before and after triggers, using OLD and NEW qualifier), Dropping a Trigger, Applications of Triggers.	10
----------	--	-----------

Topics

1. What is Trigger ?
2. Difference between Trigger and Procedure
3. Syntax for Trigger
4. HOW TO APPLY THE DATABASE TRIGGERS?
5. :OLD and :NEW Values
6. Types of Triggers
7. GUIDELINES FOR CREATING A TRIGGER
8. WHEN Clause
9. HOW TO EXECUTE A TRIGGER ?
10. How to Remove / Drop a Trigger ?
11. Trigger Examples
12. Application of Triggers

What is Trigger ?

- ***Trigger*** is a ***series of PL/SQL statements*** attached to a database table that execute whenever a ***triggering event (update, insert, delete)*** occurs.
- The ***triggers*** are ***standalone procedures*** that are ***fired implicitly (internally)*** by the oracle.
- Unlike ***stored procedures*** and ***functions***, they ***not explicitly called***, but they are activated (triggered) when a ***triggering event*** occurs.

Difference between Trigger and Procedure

- The *triggers don't accept* the *parameters* whereas *procedure* can accept.
- The oracle engine *executes* a *trigger implicitly (automatically fired)* whereas to *execute* a *procedure* it must *explicitly called by the users*.

Syntax for Trigger

Syntax to create a trigger:

```
CREATE OR REPLACE TRIGGER <TRIGGER_NAME>  
{BEFORE/AFTER}  
{INSERT/UPDATE/DELETE} [OF COLUMN] ON <TABLE_NAME>  
[FOR EACH ROW]  
[WHEN CONDITION]  
[PL/SQL BLOCK]
```

Syntax to drop a trigger:

```
DROP TRIGGER <TRIGGER_NAME>;
```

HOW TO APPLY THE DATABASE TRIGGERS?

A *trigger* has *three* basic parts. The following are the three basic parts:

1. Triggering Event or Statement:

It is a SQL statement that causes a trigger to be fired. It can be INSERT, UPDATE or DELETE statement for a specific table.

2. Trigger Restriction:

A trigger restriction specifies a Boolean Expression that must be TRUE for the trigger to fire. It is an option available for the triggers that are fired for each row. A trigger restriction is specified using a WHEN clause.

3. Trigger Action:

A trigger action is the PL/SQL code to be executed when triggering statement is encountered and the value of trigger restriction is TRUE. The PL/SQL block contains SQL and PL/SQL statements.

:OLD and :NEW Values

- When a ***DML statement*** changes a column the old and new values are visible to the executing code.
- This is done by prefixing the table column with **:old** or **:new**
- **:new** is useful for INSERT and UPDATE
- **:old** is useful for DELETE and UPDATE

Types of Triggers

1. Row Trigger
2. Statement Trigger
3. Before Trigger and After Trigger

Types of Triggers

1. *Row Trigger*

- A ***Row Trigger*** is fired ***each time*** a row in the table is ***affected*** by the triggering statement.
- For example, if the UPDATE statement updates multiple rows of a table a Row Trigger is fired once for each row affected by the UPDATE statement. If the triggering statement affects no rows, the trigger is not executes at all.

Types of Triggers

1. *Row Trigger*

```
CREATE OR REPLACE TRIGGER trig_test
AFTER UPDATE OF SNUM ON PERSONNEL
FOR EACH ROW
BEGIN
    null;                -- write operations here
END;
```



A Row trigger fires once for every row affected by the DML operation.

Types of Triggers

2. *Statement Trigger*

- A ***Statement Trigger*** is ***fired once*** on behalf of the triggering statements, independent of the number of rows the triggering statement affects (even if no rows are affected).

Types of Triggers

2. *Statement Trigger*

Inserting, Deleting and Updating can be used to find out which event is occurring

```
CREATE OR REPLACE TRIGGER trig_testTable
AFTER INSERT or UPDATE ON Personnel
BEGIN
  If Inserting
    Then INSERT into testTable values ('insert done', SYSDATE) ;
  Else
    INSERT into testTable values ('update done', SYSDATE) ;
  End If;
END;
```

Test_trigger_1 tests this trigger

Types of Triggers

3. *Before and After Trigger*

- While defining a trigger it is necessary to specify the ***trigger timing*** that we must have to specify when the ***triggering action*** is to be executed in relation to the triggering statement. ***BEFORE*** and ***AFTER*** apply to both row and the statement triggers.
 - The ***BEFORE triggers*** execute the trigger action before the triggering statement is executed.
 - The ***AFTER triggers*** executes the triggering action after the execution of triggering statement.

GUIDELINES FOR CREATING A TRIGGER:

1. There can be *only one trigger* of a particular type that is for *UPDATE, for INSERT or for DELETE*.
2. Only *one table* can be specified in the *triggering statement*.
3. The triggers *cannot include* COMMIT, ROLLBACK and SAVEPOINT statements.
4. Inside the trigger the correlation name *:NEW and :OLD* can be made use of to refer to data on the command line and data in the table respectively.

WHEN Clause

- WHEN is *optional additional statement* to control the trigger.
- Takes a **BOOLEAN** SQL expression
 - Trigger fires if **TRUE** and not if **FALSE**
- Operates on a **ROW level trigger**
- To prevent the trigger from firing in specific row cases we use **WHEN (expression)**

WHEN Example

```
create or replace trigger only_nulls
after update on BOOK
for each row
when (old.price is null) -- notice the colon with OLD is not used here
begin
    insert into PriceChange values(:old.isbn,:new.price);
end;
```

HOW TO EXECUTE A TRIGGER ?

```
SQL> START TR_EMPDELETE.SQL; OR @TR_EMPDELETE.SQL;
```

Trigger is created.....

```
SQL> DELETE FROM EMP WHERE EMPNO = 100;
```

```
SQL> SELECT * FROM NEWEMP;
```

If trigger is created with compilation error then execute the following command:

```
SQL> SHOW ERRORS;
```

To view the list of triggers created by user follow following steps:

```
SQL> DESCRIBE USER_TRIGGERS;
```

```
SQL> SELECT TRIGGER_NAME, TRIGGER_BODY FROM  
USER_TRIGGERS WHERE TRIGGER_NAME = 'TR_EMPDELETE'
```

How to Remove / Drop a Trigger ?

- **SYNTAX FOR DROPPING A TRIGGER:**

`DROP TRIGGER <TRIGGER_NAME>;`

- **SYNTAX FOR DROPPING A TRIGGER:**

`DROP TRIGGER TR_EMPUPDATE;`

Note: If conflicting triggers are created on the same database object then unwanted triggers need to be removed otherwise it will cause other trigger not to execute in a desired manner.

Examples

Example 1:

/* Write a trigger to insert the existing values of the EMP table into NEWEMP table when the record is **updated** in EMP table. (t1.sql) */

SET SERVEROUTPUT ON

CREATE OR REPLACE TRIGGER TR_EMPUPDATE
BEFORE UPDATE OF SAL ON EMP
FOR EACH ROW

BEGIN
INSERT INTO NEWEMP (EMPNO, ENAME, SAL) VALUES
(:OLD.EMPNO, :OLD.ENAME, :OLD.SAL);
END;
/

Examples

Example 2:

/* Write a trigger to insert the values into the NEWEMP table when the records are *inserted* into the EMP table. (t2.sql) */

```
SET SERVEROUTPUT ON
```

```
CREATE OR REPLACE TRIGGER TR_EMPINSERT  
BEFORE INSERT ON EMP  
FOR EACH ROW
```

```
BEGIN  
INSERT INTO NEWEMP (EMPNO, ENAME, SAL) VALUES (:NEW.EMPNO,  
:NEW.ENAME, :NEW.SAL);  
END;  
/
```

Examples

Example 3:

/* Write a trigger to insert the existing values of the EMP table into NEWEMP table when the record is *deleted* in EMP table. (t3.sql) */

```
SET SERVEROUTPUT ON
```

```
CREATE OR REPLACE TRIGGER TR_EMPDELETE  
BEFORE DELETE ON EMP  
FOR EACH ROW
```

```
BEGIN  
INSERT INTO NEWEMP (EMPNO, ENAME, SAL) VALUES  
(:OLD.EMPNO, :OLD.ENAME, :OLD.SAL);  
END;  
/
```

Examples

Example 4:

```
-- Write a trigger for INSERT, UPDATE and DELETE operation in one program (t7.sql).
SET SERVEROUTPUT ON
CREATE OR REPLACE TRIGGER TR_ALL
BEFORE INSERT OR UPDATE OR DELETE ON EMP
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO NEWEMP (EMPNO, ENAME, SAL) VALUES (:NEW.EMPNO,
:NEW.ENAME,:NEW.SAL);
    ELSIF UPDATING THEN
        INSERT INTO NEWEMP (EMPNO, ENAME, SAL) VALUES (:OLD.EMPNO,
:OLD.ENAME, :OLD.SAL);
    ELSE
        INSERT INTO NEWEMP (EMPNO, ENAME, SAL) VALUES (:OLD.EMPNO,
:OLD.ENAME, :OLD.SAL);
    END IF;
END;
/
```

Examples

Example 5:

-- Write a trigger to restrict user form using the table on Sunday.
(t10.sql)

SET SERVEROUTPUT ON

CREATE OR REPLACE TRIGGER TR_HOLIDAY

BEFORE INSERT OR UPDATE OR DELETE ON EMP

FOR EACH ROW

BEGIN

IF TRIM (TO_CHAR (SYSDATE, 'Day')) = 'Sunday' THEN

 RAISE_APPLICATION_ERROR (-20420, 'You cannot modify
data on Holiday.');

END IF;

END;

/

Examples

Example 6:

```
/* Write a trigger to restrict user form using the table on before 09:00  
am and after 05:00 pm. ( between 09 am onwards and till 5 pm )  
(t11.sql) */
```

```
SET SERVEROUTPUT ON
```

```
CREATE OR REPLACE TRIGGER TR_HOLIDAY
```

```
BEFORE INSERT OR UPDATE OR DELETE ON EMP
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF TRIM(TO_CHAR (SYSDATE, 'HH')) < 9 OR TRIM(TO_CHAR (SYSDATE,  
'HH')) >17 THEN
```

```
        RAISE_APPLICATION_ERROR (-20420, 'You cannot access the  
table before 9 am and after 5 pm');
```

```
END IF;
```

```
END;
```

```
/
```

Examples

Example 7:

-- Write a trigger to insert employee name in UPPERCASE whenever record is inserted in EMP table. (t12.sql)

SET SERVEROUTPUT ON

CREATE OR REPLACE TRIGGER trigger_upper

BEFORE INSERT ON EMP

FOR EACH ROW

BEGIN

:NEW.ENAME:=UPPER(:NEW.ENAME);

END;

/

Examples

Example 8:

-- Write a trigger that identifies the gender of the employee and according to the gender sets MR. in front of MALE employees and MS. in front of FEMALE employee.

(t13.sql)

```
SET SERVEROUTPUT ON
```

```
CREATE OR REPLACE TRIGGER TR_GENDER
```

```
BEFORE INSERT ON EMP
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF :NEW.GENDER= 'M' THEN
```

```
    :NEW.ENAME:= 'MR.' || :NEW.ENAME;
```

```
ELSIF :NEW.GENDER= 'F' THEN
```

```
    :NEW.ENAME:= 'MS.' || :NEW.ENAME;
```

```
ELSE
```

```
    :NEW.ENAME:= 'DEAR, ' || :NEW.ENAME;
```

```
END IF;
```

```
END;
```

```
/
```

Examples

Example 9:

-- Write a trigger that restricts the entry of record if salary is greater then 8000 Rs. (t14.sql)

```
SET SERVEROUTPUT ON
```

```
CREATE OR REPLACE TRIGGER TR_TESTSAL  
BEFORE INSERT OR UPDATE OF SAL ON EMP  
FOR EACH ROW
```

```
BEGIN
```

```
IF :NEW.SAL > 8000 THEN
```

```
    RAISE_APPLICATION_ERROR (-20200,'INCORRECT SALARY VALUE');
```

```
END IF;
```

```
END;
```

```
/
```

Examples

Example 10:

```
-- Write a trigger that shows the use of WHEN condition. (t4.sql)
SET SERVEROUTPUT ON
CREATE OR REPLACE TRIGGER display_salary_changes
BEFORE UPDATE ON EMP
FOR EACH ROW
WHEN (NEW.SAL > OLD.SAL)
DECLARE
    sal_diff number;
BEGIN
    sal_diff := :NEW.sal - :OLD.sal;
    dbms_output.put_line('Old salary: ' || :OLD.sal);
    dbms_output.put_line('New salary: ' || :NEW.sal);
    dbms_output.put_line('Salary difference: ' || sal_diff);
INSERT INTO NEWEMP (empno,ename,sal) VALUES (:OLD.EMPNO,
:OLD.ENAME, :OLD.SAL);
END;
/
```

Examples

Example 11:

/* Write a trigger which restrict the user from withdrawal operation if the balance amount after withdrawal operation is less then Rs. 1000. Use WHEN clause (t15.sql)*/

ACCOUNT(ACC_NO,BALANCE)

SET SERVEROUTPUT ON

CREATE OR REPLACE TRIGGER TR_BAL

BEFORE INSERT OR UPDATE OF BALANCE ON ACCOUNT

FOR EACH ROW

WHEN (NEW.BALANCE<1000)

BEGIN

 RAISE_APPLICATION_ERROR(-20650, 'Minimum balance
for account is Rs. 1000');

END;

/

Application of Triggers

- Triggers can be written for the following purposes:
 1. Generating some derived column values automatically
 2. Enforcing referential integrity
 3. Event logging and storing information on table access
 4. Auditing
 5. Synchronous replication of tables
 6. Imposing security authorizations
 7. Preventing invalid transactions